System Testing

Definition of System Testing:

System Testing is a **type of black-box testing** that evaluates the complete and fully integrated software product. It is performed **after integration testing** and before acceptance testing to verify that the entire system works as intended according to specified requirements. This phase ensures that the system meets both **functional and non-functional** requirements.

Purpose:

The main goal of system testing is to validate the end-to-end functionality of the software. It checks whether all components of the application interact properly and ensures that the application behaves as expected in real-world usage scenarios.

Important Types of System Testing (with Simple Examples):

---

1. Recovery Testing

- **Purpose**: To verify how well the system can recover from failures such as crashes, hardware malfunctions, or unexpected shutdowns.
- **Example**: If an e-book purchasing app crashes during checkout, recovery testing checks if the cart still has the selected book when the app restarts.

---

2. Security Testing

- **Purpose**: To ensure that the system is secure from unauthorized access, data leaks, and potential attacks.
- **Example**: Only registered users should be able to access their personal account and payment details. Security testing confirms that hackers cannot bypass login or access restricted areas.

---

3. Stress Testing

- **Purpose**: To determine the system's robustness by testing it under extreme workloads or resource limitations.
- **Example**: When 10,000 users try to access or download books simultaneously, stress testing checks if the app remains stable or crashes under pressure.

---

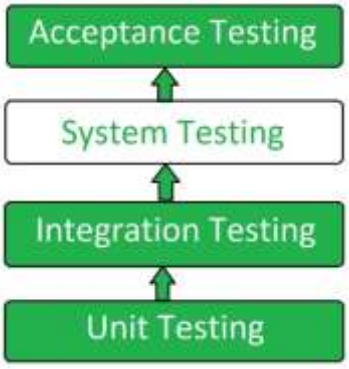
4. Performance Testing

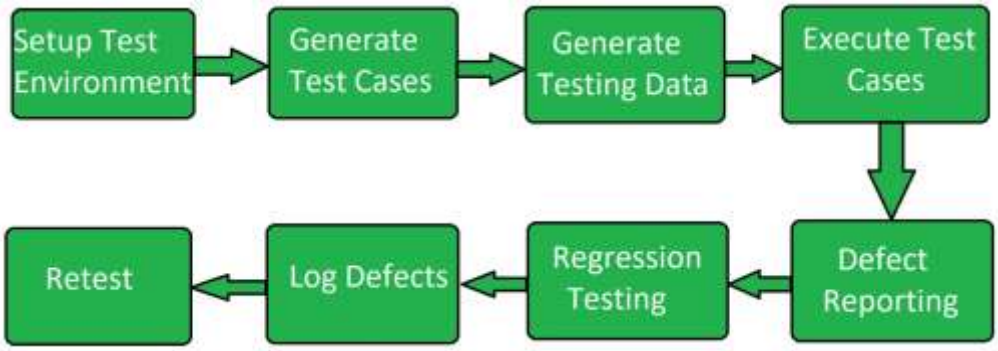
- **Purpose**: To evaluate how quickly and efficiently the system performs under a specific workload.
- **Example**: If a user searches for a book, the results should appear within 2 seconds—even if the library has thousands of entries. Performance testing ensures fast and smooth operation.

---

5. Deployment Testing

- **Purpose**: To verify that the application installs and runs correctly in the actual user environment or on target platforms.
- **Example**: Checking whether the application installs properly on Android, iOS, Windows, and Mac, and works without errors post-installation.





System Testing Process
System Testing is performed in the following steps:

- **Test Environment Setup:** Create testing environment for the better quality testing.
- **Create Test Case:** Generate test case for the testing process.
- **Create Test Data:** Generate the data that is to be tested.
- **Execute Test Case:** After the generation of the test case and the test data, test cases are executed.
- **Defect Reporting:** Defects in the system are detected.
- **Regression Testing:** It is carried out to test the side effects of the testing process.
- **Log Defects:** Defects are fixed in this step.
- **Retest:** If the test is not successful then again test is performed.

Key Characteristics of Good Software Design

A good software design ensures that the software is easy to develop, extend, maintain, and use. The following are essential characteristics that define high-quality software design:

---

1. Modularity

- **Definition**: Breaking the software into smaller, independent modules that each perform a specific function.
- **Importance**: Makes development, testing, and debugging easier.
- **Example**: In a bookstore application, login, search, and payment can be separate modules.

---

2. Maintainability

- **Definition**: The ease with which software can be modified to fix bugs or add new features.
- **Importance**: Reduces time and cost for future changes.
- **Example**: Adding a discount feature without changing the entire billing module.

---

3. Reusability

- **Definition**: Ability to use existing code or components in other applications or modules.
- **Importance**: Promotes code efficiency and saves development effort.
- **Example**: A login module reused across multiple applications.

---

4. Scalability

- **Definition**: The ability of the software to handle increasing amounts of work, such as more users or data.
- **Importance**: Ensures performance even as the software grows.
- **Example**: A web application that can support thousands of users without slowing down.

---

5. Flexibility

- **Definition**: Ease of making changes or enhancements in the design.
- **Importance**: Helps the software adapt to evolving requirements.

- **Example**: Changing a payment method without affecting other components.

---

6. Reliability

- **Definition**: The software performs its intended functions under defined conditions consistently.
- **Importance**: Builds user trust and prevents failure.
- **Example**: Orders are always placed and saved correctly in an e-commerce system.

---

7. Efficiency

- **Definition**: Optimal use of resources like time, memory, and processing power.
- **Importance**: Improves system performance and user experience.
- **Example**: Quick search results in a library app with thousands of records.

---

8. Understandability

- **Definition**: The design and code are easy for developers to read and understand.
- **Importance**: Supports better collaboration and reduces learning curve for new team members.
- **Example**: Well-structured code with meaningful variable names and comments.

---

9. Testability

- **Definition**: Ease with which the software can be tested for correctness.
- **Importance**: Ensures defects can be found and fixed early.
- **Example**: Separate modules allow for efficient unit testing.

Architectural Styles in Software Engineering

Software architecture refers to the **high-level structure** of a software system. Different architectural styles define **patterns of organization**, how components interact, and how the system is designed to perform efficiently and maintainably.

Below are four widely used architectural styles:

---

1. Layered Architecture (Tiered Architecture)

 Definition:

This style organizes the system into **hierarchical layers**, where each layer performs a specific role and communicates only with its adjacent layers.

Typical Layers:

- **Presentation Layer** – UI/UX, user interaction.
- **Business Logic Layer** – Application logic and rules.
- **Data Access Layer** – Handles database communication.
- **Database Layer** – Stores and retrieves data.

Advantages:

- Separation of concerns (clear responsibilities).
- Easier maintenance and testing.
- Supports reusability of layers.

Disadvantages:

- Performance overhead (communication between layers).
- Tightly coupled layers can reduce flexibility.

Examples:

- Enterprise web applications (Java EE, .NET)
- MVC Frameworks (Model-View-Controller)

---

2. Client-Server Architecture

 Definition:

This style divides the system into **two roles**:

- **Client**: Requests services (front-end or user interface).

- **Server**: Provides services (data processing, storage).

How It Works:

Clients send requests to the server; the server processes and returns results.

Advantages:

- Centralized control and security.
- Easy to upgrade the server.
- Efficient resource sharing.

Disadvantages:

- Server is a single point of failure.
- Network dependency can affect performance.

Examples:

- Web browsers and web servers.
- Online banking, email systems.

---

3. Data Flow Architecture (Pipe and Filter / Batch Sequential)

 Definition:

This style represents systems as a **series of processing elements (filters)** connected by **data channels (pipes)**. Each filter transforms input data and sends it to the next.

Variants:

- **Batch Sequential**: Components execute one after another on entire data sets.
- **Pipe & Filter**: Continuous data flows through a series of components.

Advantages:

- Supports reusability of filters.
- Easy to add or replace components.
- Good for data transformation systems.

Disadvantages:

- Not ideal for interactive systems.
- Error handling between filters can be complex.

Examples:

- Compilers (tokenizing → parsing → code generation).

- Unix shell pipelines (`cat file | grep "error" | sort`)

---

4. Call and Return Architecture (Procedural Architecture)

Definition:

This style uses **function calls and returns** to manage control flow. A main program calls different modules/subroutines which may call others in a hierarchical manner.

Structure:

- **Main Program** – Controls execution.
- **Submodules** – Perform specific tasks and return control.

Advantages:

- Simple and well-supported in procedural programming.
- Good for structured, smaller applications.

Disadvantages:

- Difficult to scale for large systems.
- Tight coupling reduces flexibility.

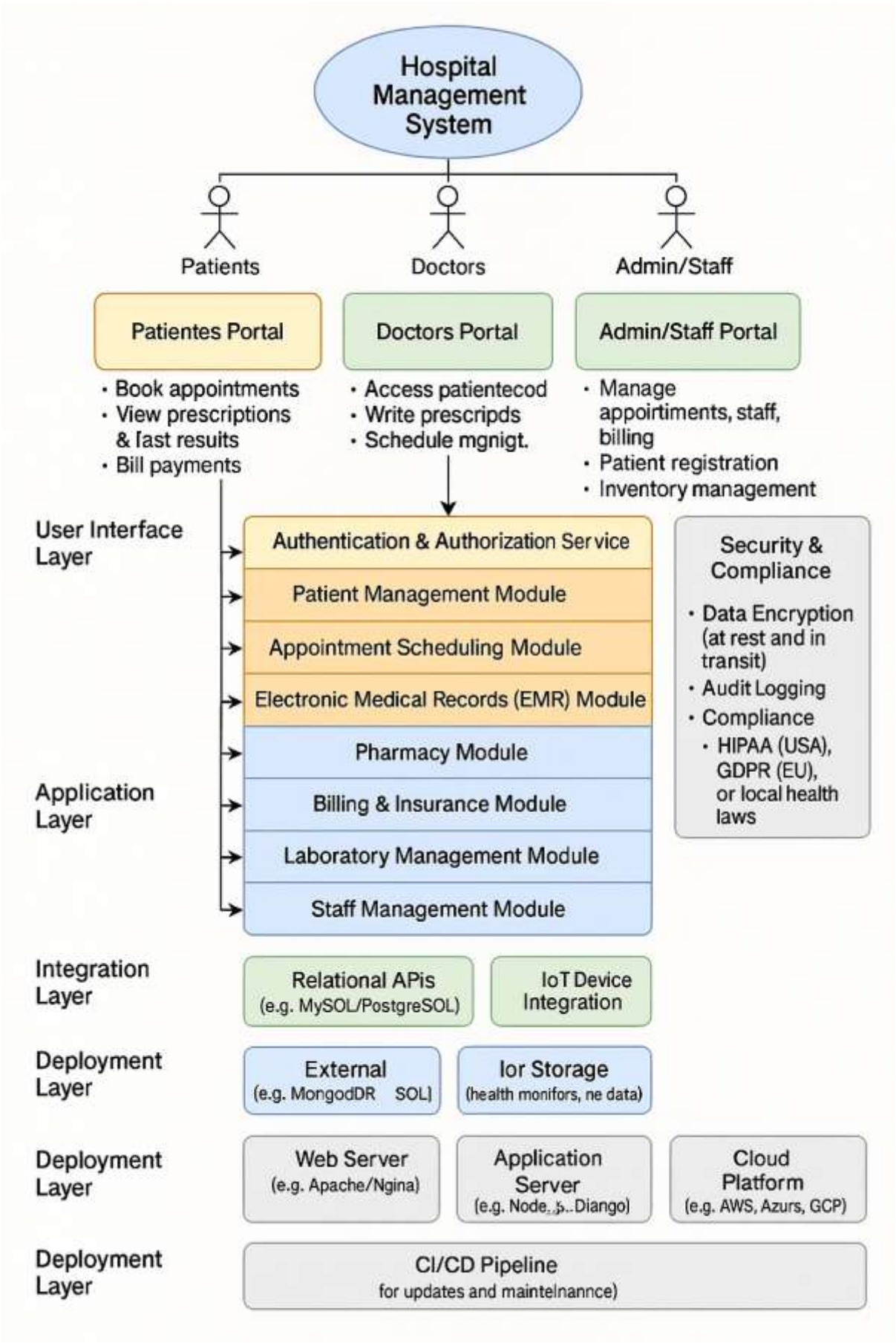
Examples:

- C/C++ programs with functions and procedures.
- Legacy monolithic systems.

Conclusion:

Choosing the right architectural style is vital for building **scalable, maintainable, and efficient software**.

- **Layered** is best for modular systems.
- **Client-Server** suits distributed systems.
- **Data Flow** is ideal for data processing.
- **Call and Return** is good for simple, procedural programs.


Hospital Management System
Patients
Doctors
Admin/Staff
Patientes Portal
Doctors Portal
Admin/Staff Portal
• Book appointments
• View prescriptions & last resuits
• Bill payments
• Access patientecod
• Write prescripds
• Schedule mgnigt.
• Manage appoirtiments, staff, billing
• Patient registration
• Inventory management
User Interface Layer
Authentication & Authorization Service
Patient Management Module
Appointment Scheduling Module
Electronic Medical Records (EMR) Module
Pharmacy Module
Billing & Insurance Module
Laboratory Management Module
Staff Management Module
Application Layer
Security & Compliance
• Data Encryption (at rest and in transit)
• Audit Logging
• Compliance
• HIPAA (USA), GDPR (EU), or local health laws
Integration Layer
Relational APis (e.g. MySOL/PostgreSOL)
IoT Device Integration
Deployment Layer
External (e.g. MongodDR SOL)
Ior Storage (health monifors, ne data)
Deployment Layer
Web Server (e.g. Apache/Ngina)
Application Server (e.g. Node..ʃ..Diango)
Cloud Platform (e.g. AWS, Azurs, GCP)
Deployment Layer
CI/CD Pipeline for updates and maintelnannce)

High-Level Architectural Diagram of Hospital Management System

The provided diagram illustrates a **high-level architecture** of a **Comprehensive Hospital Management System (HMS)**, structured in layers and components that represent the functioning of a modern digital hospital ecosystem.

---

1. User Roles & Portals (Top Layer)

This architecture identifies **three primary user roles**:

- **Patients**: Can book appointments, view prescriptions and results, and make bill payments.
- **Doctors**: Access patient records, prescribe treatments, and manage schedules.
- **Admin/Staff**: Manage appointments, billing, patient registration, and inventory.

Each role accesses the system through **dedicated portals**, ensuring tailored access based on needs and privileges.

---

2. User Interface Layer

This layer provides interactive **portals** for each user type. It ensures intuitive navigation and secure communication with the backend services.

---

3. Application Layer

The core business logic resides here, made up of modular components:

- **Authentication & Authorization Service**: Manages secure login and role-based access control.
- **Patient Management Module**: Stores and retrieves patient profiles.
- **Appointment Scheduling Module**: Handles doctor-patient meeting schedules.
- **Electronic Medical Records (EMR)**: Digitally maintains patient health records.
- **Pharmacy Module**: Handles prescription management and drug inventory.
- **Billing & Insurance Module**: Automates payment and insurance processes.
- **Laboratory Management Module**: Manages test scheduling and report handling.
- **Staff Management Module**: Organizes internal staff operations and HR data.

---

4. Security & Compliance

This is a crucial component:

- **Data encryption** ensures patient data is protected during transmission and storage.
- **Audit logging** keeps track of sensitive operations.
- Ensures legal **compliance** with laws like **HIPAA**, **GDPR**, or regional regulations.

---

5. Integration Layer

Ensures connectivity with:

- **Relational APIs** (MySQL/PostgreSQL) for structured data operations.
- **IoT Devices** (e.g., patient monitors) to enhance real-time healthcare data collection.

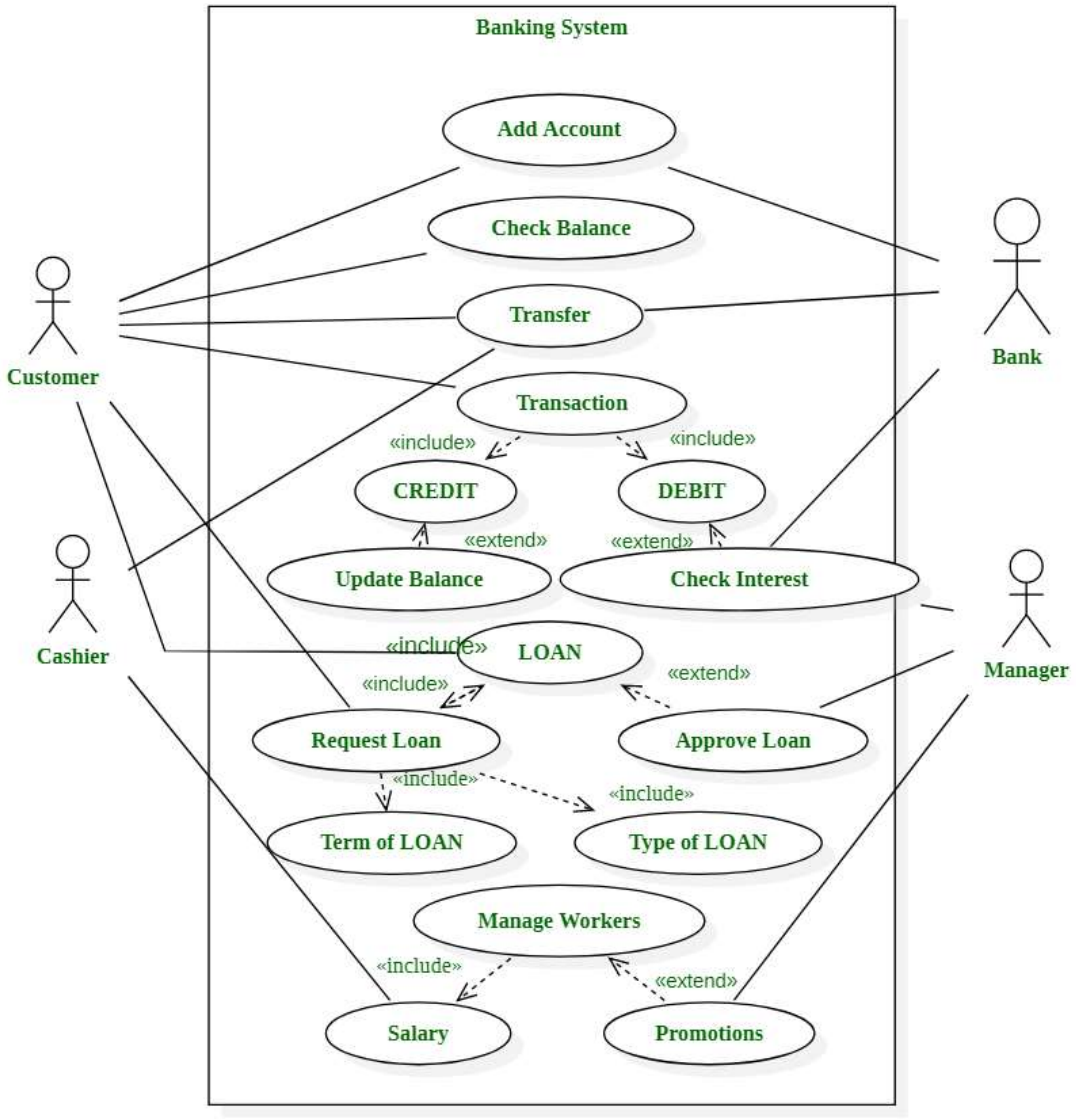
---

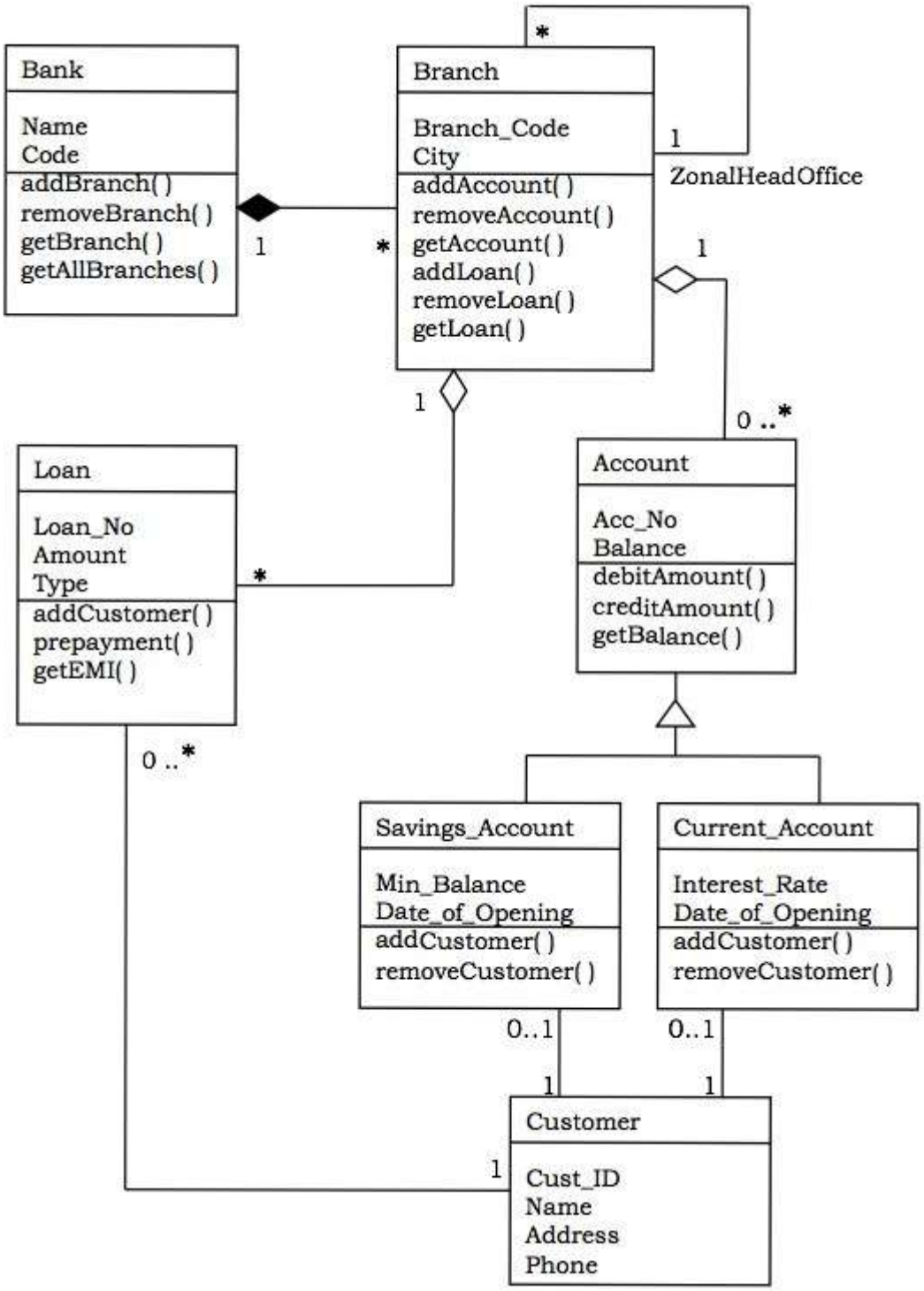
6. Deployment Layer

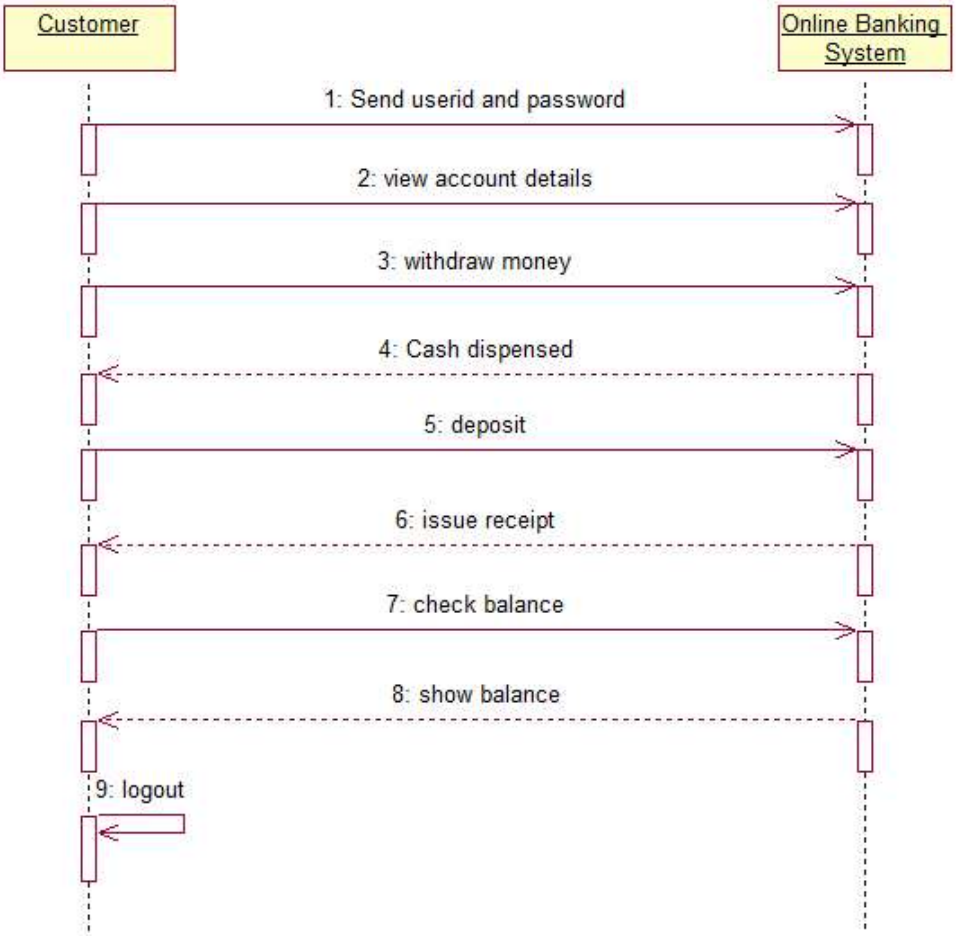
This consists of several critical infrastructure components:

- **External and local storage** for unstructured data like reports.
- **Web Server** (e.g., Apache, Nginx) for handling HTTP requests.
- **Application Server** (e.g., Node.js, Django) to run backend logic.
- **Cloud Platform** (AWS, Azure, GCP) for scalable hosting.
- **CI/CD Pipeline** for seamless software updates and maintenance.

ONLINE BANKING SYSTEM




Bank
Name
Code
addBranch( )
removeBranch( )
getBranch( )
getAllBranches( )
Branch
Branch_Code
City
addAccount( )
removeAccount( )
getAccount( )
addLoan( )
removeLoan( )
getLoan( )
ZonalHeadOffice
Loan
Loan_No
Amount
Type
addCustomer( )
prepayment( )
getEMI( )
Account
Acc_No
Balance
debitAmount( )
creditAmount( )
getBalance( )
Savings_Account
Min_Balance
Date_of_Opening
addCustomer( )
removeCustomer( )
Current_Account
Interest_Rate
Date_of_Opening
addCustomer( )
removeCustomer( )
Customer
Cust_ID
Name
Address
Phone
1
*
0 ..*
0..1



https://www.geeksforgeeks.org/online-railway-ticket-reservation-system/